



Distributed Systems

Mutual Exclusion



Recap



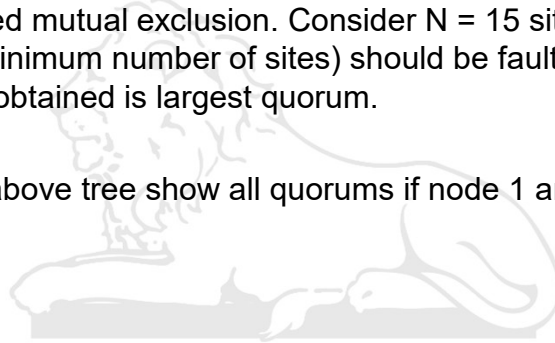
- Maekawa's Algorithm
- Agarwal-El Abbadi Quorum-Based Algorithm



Agarwal-El Abbadi Quorum-Based Algorithm



1. Consider Agrawal-El Abbadi quorum-based algorithm for distributed mutual exclusion. Consider $N = 15$ sites. How many (minimum number of sites) should be faulty so that the quorum obtained is largest quorum.
2. For the above tree show all quorums if node 1 and 2 are down.



IIT ROORKEE ■ ■ ■

Distributed Mutual Exclusion Algorithms

- **Non-token based / Permission based**
 - Permission from all processes: e.g. Lamport, Ricart-Agarwala etc.
- **Quorum based**
 - Permission from a subset: ex. Maekawa
- **Token based**
 - ex. Suzuki-Kasami, Raymond's tree based
- Tradeoff: Synchronization delay (or throughput) vs message traffic

Token-Based Algorithms

- In token-based algorithms, a unique token is shared among the sites.
- A site is allowed to enter its CS if it possesses the token.
- Token-based algorithms use sequence numbers instead of timestamps. (Used to distinguish between old and current requests.)
- No need for correctness proof
- Issues such as freedom from starvation and deadlock are prominent

Suzuki-Kasami's Broadcast Algorithm

- If a site wants to enter the CS and it does not have the token, it broadcasts a REQUEST message for the token to all other sites.
- A site which possesses the token sends it to the requesting site upon the receipt of its REQUEST message if
 - It is not executing the CS else it sends the token after it has completed the execution of the CS
- A site holding the token can enter into CS repeatedly

Scenario 1

1. j and k wants to enter in CS and i has token
2. i receives the request of j and gives token
3. j completes CS and passes token to k
4. k completes CS
5. i request for token
6. k gives token to i
7. i enters in CS
8. Request of k (old) arrives

Design issues

1. How to distinguish an **outdated REQUEST message** from a current REQUEST message.
 - If a site can not determine that the request corresponding to a token has been satisfied, it may dispatch the token to a site that does not need it.
 - This will not violate the correctness, however, this may seriously degrade the performance.
2. How to determine which site has an **outstanding request for the CS**
 - so that the token can be dispatched to one of them

Handling Outdated / Outstanding requests

- A REQUEST message of site S_j has the form
 $\text{REQUEST}(j, n)$
 where n is a seq number which indicates that site S_j is requesting its n th CS execution.
- A site S_i keeps an array of integers $\text{RN}_i[1..N]$ where $\text{RN}_i[j]$ denotes the largest sequence number received in a REQUEST message so far from site S_j .
- When site S_i receives a $\text{REQUEST}(j, n)$ message
 - set $\text{RN}_i[j] := \max(\text{RN}_i[j], n)$
 - ⇒ discard if $\text{RN}_i[j] > n$

Handling Outdated / Outstanding requests

- The token consists of
 - a queue of requesting sites, Q , and
 - an array of integers $\text{LN}[1..N]$, where $\text{LN}[j]$ is the sequence number of the request which site S_j executed most recently.
- After executing its CS, a site S_i updates

$$\text{LN}[i] := \text{RN}_i[i]$$
- At site S_i if $\text{RN}_i[j] = \text{LN}[j] + 1$, then site S_j is currently requesting token.

Suzuki-Kasami's Broadcast Algorithm

Requesting the critical section

- (a) If requesting site S_i does not have the token, then
 - it increments its sequence number, $RN_i[i]$, and
 - sends a REQUEST(i, sn) to all other sites.
(‘sn’ is the updated value of $RN_i[i]$.)
- (b) When a site S_j receives this message
 - it sets $RN_j[i]$ to $\max(RN_j[i], sn)$.
 - If S_j has the idle token, it sends the token to S_i if
 $RN_j[i] = LN[i] + 1$.

Executing the critical section

- Site S_i executes the CS after it has received the token.

Releasing the critical section (site S_i)

- Set $LN[i]$ element of the token array equal to $RN_i[i]$.
- For every site S_j whose id is not in the token queue, append its id to the token queue if
 $RN_i[j] = LN[j] + 1$.
- If the token queue is nonempty after the above update
 - Delete the top site id from the token queue and send the token to the site indicated by the id.

Let there be 5 sites

$RN1 = RN2 = RN3 = RN4 = RN5 = [5\ 3\ 4\ 2\ 7]$

Site 2 has token and is in CS

Site 5 requests

Site 1 requests

Site 3 requests

Token Q is empty

$LN = [5\ 2\ 4\ 2\ 7]$

Let there be 5 sites

$RN1 = RN2 = RN3 = RN4 = RN5 = [3\ 4\ 4\ 2\ 8]$

Site 2 has token and is in CS

$LN = [5\ ?\ 4\ 2\ 7]$

Correctness

Mutual exclusion is guaranteed because there is only one token in the system and a site holds the token during the CS execution.

Performance

No. of messages per CS execution: 0 to N

Synchronization delay: 0 or T (high load)

Response Time (low load): $2T + E$

Theorem: A requesting site enters the CS in finite time.

Proof: 1. Token request messages of a site S_i reach other sites in finite time.

2. Since one of these sites will have token in finite time, site S_i 's request will be placed in the token queue in finite time.

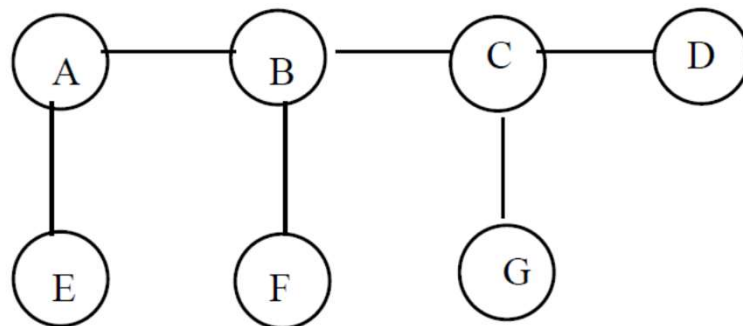
3. Since there can be at most $N - 1$ requests in front of this request in the token queue, site S_i will get the token and execute the CS in finite time.

Raymond's Tree-Based Algorithm

- Uses a spanning tree to **reduce the number of messages exchanged per critical section execution.**
- The network is viewed as a graph
- A spanning tree of a network is logically created.
- Assume
 - The underlying network guarantees message delivery.
 - All nodes of the network are completely reliable.

Raymond's Tree-Based Algorithm

- Sites are logically arranged as an **unrooted (directed / undirected) tree**



Raymond's Tree-Based Algorithm

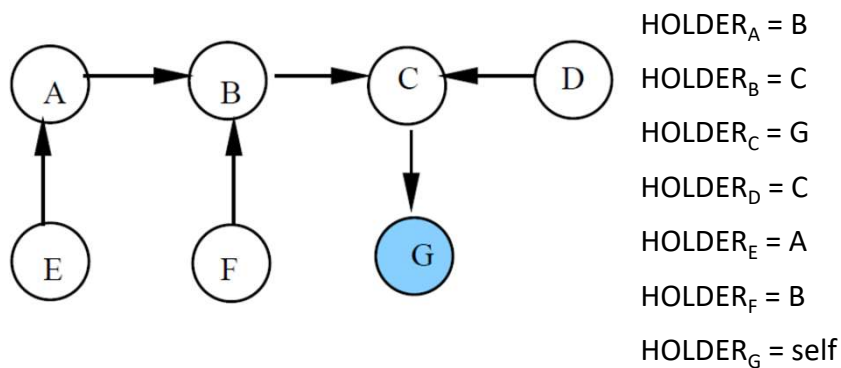
- Sites are logically arranged as a **directed tree**
 - => Edges of the tree are assigned direction towards the node holding token
- **There is no need for each node to be aware of the tree as a whole**
 - => A node needs to hold information about and communicate only to its immediate neighbor nodes
- Message between the nodes traverse along the **undirected edges** of the tree

Raymond's Tree-Based Algorithm

- Instead of token the algorithm uses the concept of **privilege**
- Only one node can be in the possession of the privilege (privileged node) at any time except when the privilege is in transit (in the form of a PRIVILEGE message)
- When there are no nodes requesting for the privilege it remains with last node that used it

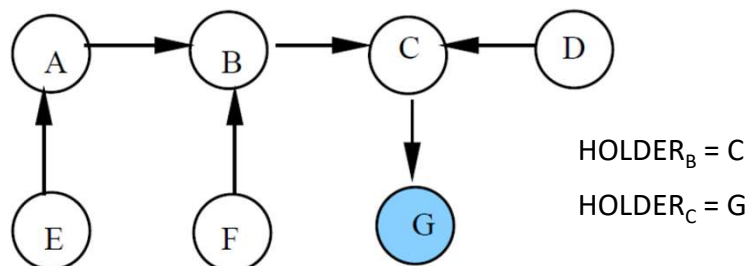
Raymond's Tree-Based Algorithm

- Each node maintains a HOLDER variable that stores the identity of a node that it thinks has the privilege or leads to the node having the privilege



The HOLDER Variables

- When node B wants to execute the critical section.



- B sends a REQUEST message to $HOLDER_B$, i.e., C, which in turn forwards the REQUEST message to $HOLDER_C$, i.e., G.

The HOLDER Variables

- The privileged node G, if it no longer needs the privilege,
 - sends the PRIVILEGE message to its neighbor C, which made a request for the privilege, and
 - sets $HOLDER_G$ to C.
- Node C forwards the PRIVILEGE to node B, since it had requested the privilege on behalf of B.
- Node C also resets $HOLDER_C$ to B.

Data Structures

Each node maintains the following variables:

Variable Name	Possible Values	Comments
HOLDER	"self" or the identity of one of the immediate neighbours.	Indicates the location of the privileged node in relation to the current node.
USING	True or false.	Indicates if the current node is executing the critical section.
REQUEST_Q	A FIFO queue that could contain "self" or the identities of immediate neighbors as elements.	The REQUEST_Q of a node consists of the identities of those immediate neighbors that have requested for privilege but have not yet been sent the privilege.
ASKED	True or false.	Indicates if node has sent a request for the privilege.

Data Structures

- The value “self” is placed in REQUEST_Q if the node makes a request for the privilege for its own use.
- ASKED prevents
 - the sending of duplicate requests for privilege, and also
 - makes sure that the REQUEST_Qs of the various nodes do not contain any duplicate elements.

The Algorithm

The algorithm consists of the following routines:

ASSIGN PRIVILEGE

MAKE REQUEST

ASSIGN PRIVILEGE:

- This routine sends a PRIVILEGE message.
- A privileged node sends a PRIVILEGE message if
 - it holds the privilege but is not using it,
 - its REQUEST_Q is not empty, and
 - the element at the head of its REQUEST_Q is not “self.”

ASSIGN-PRIVILEGE:

```

if HOLDER = self  $\wedge$   $\neg$ USING  $\wedge$  REQUEST_Q  $\neq$  empty
then
    HOLDER := dequeue (REQUEST_Q)
    ASKED := false
    if HOLDER = self
    then
        USING := true
        (initiate entry into critical section)
    else
        send PRIVILEGE to HOLDER

```

MAKE REQUEST

- This is routine sends a REQUEST message.
- An unprivileged node sends a REQUEST message if
 - it does not hold the privilege,
 - its REQUEST_Q is not empty, i.e., it requires the privilege for itself, or on behalf of one of its immediate neighboring nodes, and
 - it has not sent a REQUEST message already.

MAKE- REQUEST:

if (**HOLDER** $\neq$ **self** $\wedge$ REQUEST_Q $\neq$ empty
 $\wedge$ **ASKED**)

then

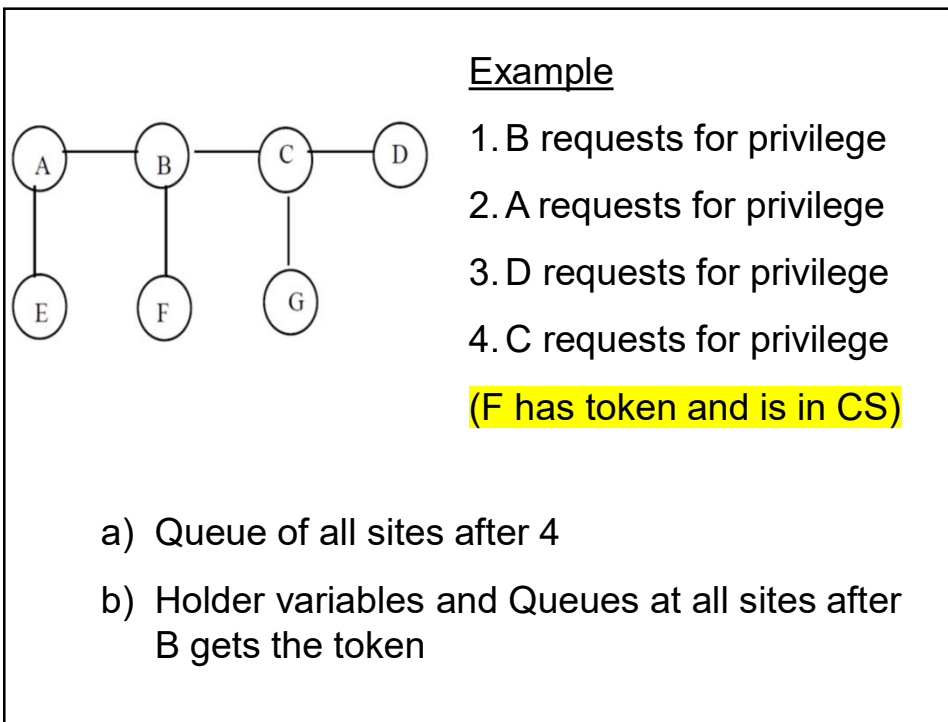
send REQUEST to HOLDER

ASKED := true

- REQUEST_Q of any node is bounded, even when operating under heavy load.

Events

Event	Algorithm Functionality
A node wishes to execute critical section.	Enqueue(REQUEST_Q, self); AS- SIGN_PRIVILEGE; MAKE_REQUEST
A node receives a REQUEST message from one of its immediate neighbors X.	Enqueue(REQUEST_Q, X); AS- SIGN_PRIVILEGE; MAKE_REQUEST
A node receives a PRIVILEGE message.	HOLDER := self; ASSIGN_PRIVILEGE; MAKE_REQUEST
A node exits the critical section.	USING := false; ASSIGN_PRIVILEGE; MAKE_REQUEST



Logical pattern of message flow between neighboring nodes A and B.

Node A	<-----	REQUEST	-----	Node
Node A	-----	PRIVILEGE	----->	Node
Node A	-----	REQUEST	----->	Node
Node A	<-----	PRIVILEGE	-----	Node
		●		
		●		
		●		
		<pattern repeats>		
		●		

Message Overtaking

- No use of sequence numbers.
- Message overtaking may occur between the nodes A and B, when
 - a PRIVILEGE message is sent from node A to node B
 - which is then very closely followed by a REQUEST message from node A to node B
- Such a message overtaking will not affect the operation of the algorithm.

Message Overtaking

Case 1: Node B receives the PRIVILEGE message from node A before receiving the REQUEST message from A

- When node B receives the PRIVILEGE message
 - it could enter the critical section or
 - could send a PRIVILEGE message to an immediate neighbor at the head of REQUEST_Q_B
- If REQUEST_Q_B is not empty; send request
- If B receives A's request it will be queued in REQUEST_Q_B

Message Overtaking

Case 2: Node B receives the REQUEST message from node A before receiving the PRIVILEGE message from A

- A's request will be queued in REQUEST_Q_B
- Since B is not a privileged node, it will not be able to send a privilege to node A in reply.
- When node B receives the PRIVILEGE message
 - it could enter the critical section or
 - could send a PRIVILEGE message to an immediate neighbor at the head of REQUEST_Q_B, which need not be node A.

Correctness

The algorithm provides the following guarantees:

- Mutual exclusion is guaranteed
- Deadlock is impossible
- Starvation is impossible